Episode 2: Building a Network Data Model from Scratch

Video Title

"Network as Code: Defining Your Entire Fabric in YAML"

Target Length

18-22 minutes

Goal

Show viewers what a network data model looks like, why it matters, and how schema validation catches mistakes before they ever touch a router. Demonstrate the concept without walking through the full build process.

Pre-Recording Checklist

Before you hit record, make sure:
- ContainerLab topology is running (all 6 devices UP)
- Have a text editor open with the data/ directory ready
- Terminal ready to run validation commands
- Prepare a "broken" version of a data file for the demo (overlapping subnet or missing VRF reference)

SEGMENT 1: Recap and Context (0:00 - 1:30)

What to show: Quick flash of the fabric status from Episode 1.

```
uv run python -m scripts.fabric_status
```

Talking points:
- "Last episode I showed you a fully automated spine-leaf fabric. Six devices, OSPF underlay, iBGP EVPN overlay, all deployed from code. Today we start building that from the ground up."
- "Every automation project starts with the same question: where does the truth about your network live? If it lives in device configs scattered across 200 switches, you do not have a source of truth. You have 200 sources of opinion."
- "Lab 1 solves that by putting every fact about the network into structured YAML files with enforced schemas. That is the data model."

SEGMENT 2: Why a Data Model (1:30 - 4:00)

What to show: Nothing on screen, or a simple slide with "Intent vs Device-Centric" comparison.

Talking points:
- "There are two ways to describe a network in code. Device-centric and intent-based."
- "Device-centric means you write the config for each device individually. Spine1 gets these BGP neighbors. Leaf1 gets those BGP neighbors. If you add a new spine, you have to update every single leaf config by hand."
- "Intent-based means you declare the design. 'Spine1 is a route reflector. All leafs peer with all spines.' The automation figures out what that means for each device. Add a third spine, and every leaf config updates automatically."
- "The data model is how you express that intent. It is not configuration. It is a description of what the network should look like."
- "Cisco's Network as Code paper calls this the single source of truth. And they found that 80% of network outages come from the gap between what engineers intended and what actually got configured."

SEGMENT 3: The Data Model Structure (4:00 - 8:00)

What to show: The file tree of the data/ directory. Do NOT open any files fully.

```
ls -la data/
ls -la data/services/
```

Walk through each file's purpose without showing contents:

- "fabric.yaml holds the fabric-wide settings. The AS number, the underlay routing protocol, global defaults. Things that apply to every device."
- "topology.yaml is the device inventory. Every device, its role, its loopback address. This is where the automation knows that spine1 exists and that it is a spine."
- "underlay.yaml defines the point-to-point links between devices and the OSPF parameters. Every fabric link, both ends, with IP addressing."
- "overlay.yaml is the iBGP EVPN configuration. Which devices are route reflectors, the address families, the peering model."
- "Then under services, you have vrfs.yaml and networks.yaml. These are the network services riding on top of the fabric. VRFs, subnets, VXLAN network identifiers."

Talking points:
- "Notice the separation. Fabric facts live in one place. Services live in another. If you want to add a new VRF, you do not touch the underlay configuration. You add an entry to vrfs.yaml."
- "This is the same principle behind any well-designed system. Separate the concerns. Change one thing without breaking another."

SEGMENT 4: Intent vs Device Config (8:00 - 11:00)

What to show: A quick comparison. Show ONE small snippet from the data model (just the route reflector flag, not the full file), then show what it produces in the generated config.

Option A: Show the topology entry for spine1 with route_reflector: true (just that one line, maybe 3-4 lines of context)

Then show the generated BGP config:

```
grep -A5 "neighbor" configs/spine1/frr.conf | head -20
```

Talking points:
- "In the data model, spine1 has one flag: route_reflector: true. That is it. One line."
- "But look at the generated config. Every leaf device appears as a BGP neighbor with route-reflector-client set. The automation built all of that from a single boolean."
- "Now imagine changing that flag to false. Or adding it to spine2. Every device's BGP config changes automatically. That is what intent-based means in practice."

Then show the reverse: What the same thing looks like without a data model.
- "Without the data model, you would be editing six separate config files every time you change the route reflector design. And hoping you did not miss one."

SEGMENT 5: Schema Validation Demo (11:00 - 15:00)

What to show: Break something in the data model, run validation, watch it fail.

Demo 1: Type Validation

Do NOT show the full schema code. Just show the result.

Exact edit: In data/services/networks.yaml, change line 11 from vlan_id: 10 to vlan_id: 5000

```
uv run python validate.py
```

- "The syntax validator caught that immediately. It knows a VLAN ID has to be between 1 and 4094. It knows an ASN has to be a positive integer. These rules are defined in Pydantic schemas, and they enforce the data model's contract."

Revert: change vlan_id: 5000 back to vlan_id: 10

Demo 2: Semantic Validation

Exact edit: In data/services/networks.yaml, change line 14 from vrf: PRODUCTION to vrf: STAGING

```
uv run python validate.py
```

- "SEM-04. The semantic validator found a reference to a VRF that is not defined. This is a cross-file check. The networks file references a VRF, and the validator confirms that VRF actually exists in vrfs.yaml."
- "This is the kind of mistake that a YAML linter can never catch. The YAML is perfectly valid. The syntax is correct. But the logic is broken. Without semantic validation, you would deploy this and get an error on the device. Or worse, silent misconfiguration."

Revert: change vrf: STAGING back to vrf: PRODUCTION

SEGMENT 6: Pydantic in 60 Seconds (15:00 - 16:30)

What to show: Just a quick flash of the schemas directory. Do NOT walk through the code.

```
ls schemas/
```

Talking points:
- "The schemas are written in Pydantic. If you have not used it before, think of it as Python's way of saying 'this data must look exactly like this.' You define a model, and Pydantic enforces it."
- "Every field has a type, constraints, and optionally a default value. If the data does not match, you get a clear error telling you exactly what is wrong and where."
- "I am not going to walk through the schema code in this video. That is in the build guide. But I want you to understand that this is not just 'check if the YAML parses.' This is a full contract for what valid network intent looks like."

SEGMENT 7: The defaults.yaml Pattern (16:30 - 18:00)

What to show: Mention defaults.yaml without showing its full contents.

Talking points:
- "One more concept worth mentioning: defaults. In a real fabric, 90% of your devices share the same OSPF timers, the same MTU, the same interface descriptions. Writing that on every device entry would be repetitive and error-prone."
- "defaults.yaml holds those shared values. Individual device entries can override them when needed, but you only define the common case once."
- "This is the DRY principle. Don't repeat yourself. And it means when you need to change the OSPF hello timer across the entire fabric, you change it in one place."

SEGMENT 8: The Close (18:00 - 20:00)

What to show: Back to the fabric status or the series overview.

Talking points:
- "That is Lab 1. The data model is the foundation that everything else builds on. Validation, config generation, CI/CD, drift detection, AI integration. All of it reads from this same set of YAML files."
- "If you want to build this yourself, the full lab guide walks you through creating every file, writing the Pydantic schemas, and setting up the validation. That is available at [your link] as part of the $49 lab package."
- "Next episode, we go deeper on validation. Four layers of checks, 39 rules, and I will show you how the compliance validator encodes your organization's policies as code."
- "Subscribe, and I will see you in the lab."

Commands Reference (Quick Copy)

```
# Fabric status
uv run python -m scripts.fabric_status

# List data files
ls data/
ls data/services/

# Run validation
uv run python validate.py

# Grep BGP neighbors from generated config
grep -A5 "neighbor" configs/spine1/frr.conf | head -20

# List schemas
ls schemas/
```

Do NOT Show On Camera

- Full contents of any YAML data file (paid content)
- The Pydantic schema code in detail (paid content)
- Step-by-step process of creating the data model (paid content)
- The defaults.yaml file contents (paid content)

DO Show On Camera

- The file tree (ls output of data/ and schemas/)
- One or two lines in context (route_reflector flag) to illustrate intent
- Validation output (errors and success messages)
- Generated config snippets (grep output, not full files)
- The concepts: intent vs device-centric, schema validation, defaults pattern